

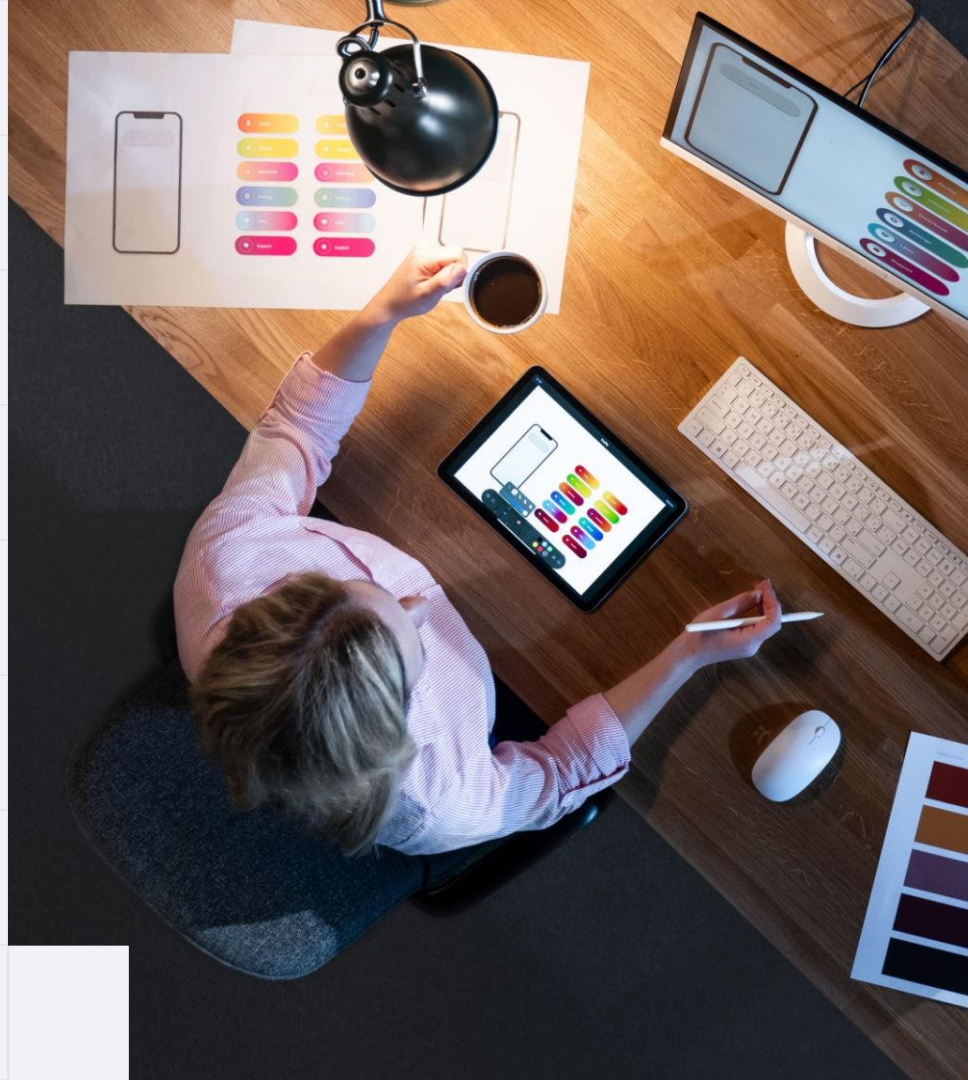


STL

Standard Template Library

What is STL in C++?

STL (Standard Template Library) is a **collection** of ready-made **data structures and algorithms** that you can use instead of writing everything from scratch.



Components of STL



Containers

Hold and organize the data.



Algorithms

Perform actions like sorting or searching on the data.



Iterators

Helps go through the data in containers one by one.

Components of STL

Data structures
(called
Containers)

vector
list
stack
queue
map
set
and more...

Algorithms

sort()
find()
reverse()
max_element(),
min_element(),
etc.

Iterators

Special
objects
used to
loop
through
containers.

Why to use STL?

- It saves time (no need to write your own stack, vector, queue, etc.)
- It is optimized and fast
- It is easy to use
- It works with any data type (templates)

Vector

Vector is a **dynamic array**.

Syntax of Declaring Vector:

```
vector< object_type > vector_variable_name;
```

```
vector< int > v1;
```

```
vector< double > v2 ;
```

```
vector< int > v3(100);
```

```
vector< int > v4 (100, -1);
```

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- `erase()`
- `insert()`

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- `erase()`
- `insert()`

returns the number of items the vector presently contains.

```
vector< int > v1;
```

```
int n = v1.size();
```


Vector

Commonly Used Member Functions

- size()
- **push_back(x)**
- pop_back()
- clear()
- empty()
- begin()
- end()
- erase()
- insert()

Push elements at the end of the vector.

```
vector< int > v1;
```

```
v1. push_back(10);  
v1. push_back(15);  
v1. push_back(2);
```

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- `erase()`
- `insert()`

Removes the last element from the vector.

```
v1.pop_back();
```

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- **`clear()`**
- `empty()`
- `begin()`
- `end()`
- `erase()`
- `insert()`

Removes all the element from the vector.

```
v1. clear();
```

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- **`empty()`**
- `begin()`
- `end()`
- `erase()`
- `insert()`

Returns true if the vector is empty.

```
v1. empty();
```

Vector

Commonly Used Member Functions

- size()
- Push back(x)
- Pop back()
- clear()
- Empty()
- **begin()**
- end()
- erase()
- insert()

Returns a pointer that points to the first element .

*v1. **begin()**;*

```
int main() {  
    vector<int> v1 = {1, 3, 5, 2, 4};  
    cout << *(v1.begin());  
    return 0;  
}
```

Output: 1

Vector

Commonly Used Member Functions

- size()
- Push back(x)
- Pop back()
- clear()
- empty()
- begin()
- **end()**
- erase()
- insert()

It's primarily used as the stopping condition in loops, immediately after the last actual item.

`v1. end();`

```
int main() {  
    vector<int> v = {1, 3, 5, 2, 4};  
    cout << *(v.end()); // Output: 4  
    cout << v.back(); // Output: 4  
    return 0;  
}
```

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- **`erase()`**
- `insert()`

It removes an element of a specific position or range of elements from the vector.

// Remove single element

`v1.erase(v1.begin() + 1);` //remove at position 1

// Erase range of elements

`v1.erase(v.begin() + 1, v.begin() + 4);`

Vector

Commonly Used Member Functions

- `size()`
- `push_back(x)`
- `pop_back()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- `erase()`
- **`insert()`**

Used to insert new elements at the given position in a vector.

`v1.insert(position, value);`

```
vector<int> v1 = {1, 2, 4, 5};
```

```
// Insert element 3 at index 2
```

```
v1.insert(v1.begin() + 2, 3); //output: 1 2 3 4 5
```

```
// Insert List of Elements
```

```
v1.insert(v1.begin() + 4, {4, 5}); //output: 1 2 3 4 5 4 5
```



```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v; // a vector of integers

    // Adding elements
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    // Printing elements
    cout << "Vector elements: ";
    for(int i=0; i<v.size(); i++) {
        cout << v[i] << " ";
    }
    cout << endl;
```

```
// Accessing an element
cout << "First element: " << v[0] << endl;

// Removing last element
v.pop_back();

cout << "After pop_back(): ";
for(int x : v) {
    cout << x << " ";
}
cout << endl;
```

Stack

Stack container follows LIFO (Last In First Out) order of insertion and deletion.

Commonly Used Operations in Stack

- `empty()`
- `size()`
- `push(x)`
- `pop()`
- `top()`

Stack

Commonly Used operations in stack

- `empty()`
- `size()`
- `push(x)`
- `pop()`
- `top()`

Checks whether the stack is empty.

It returns true if the stack has no elements; otherwise, it returns false.

```
st1.empty();
```

Stack

Commonly Used operations in stack

- empty()
- size()
- push(x)
- pop()
- top()

Returns the number of elements currently in the stack.

```
st1.size();
```

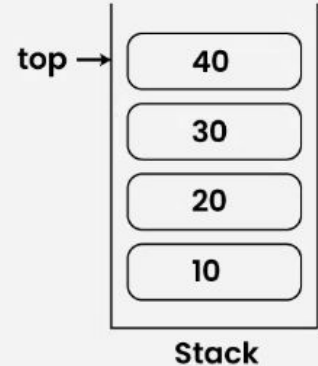
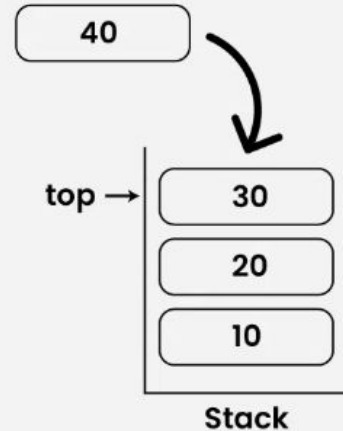
Stack

Commonly Used operations in stack

- empty()
- size()
- **push(x)**
- pop()
- top()

In stack, new elements can only be inserted at the top of the stack by using push(x) .

st1.**push**(40);



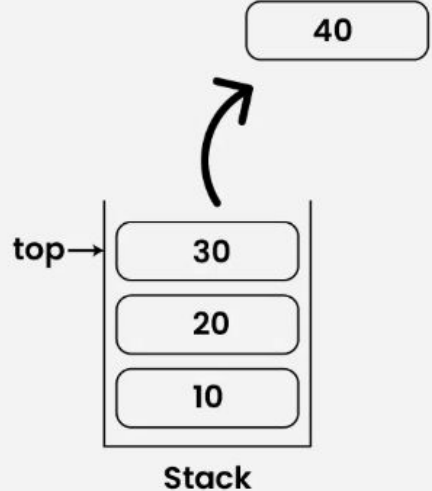
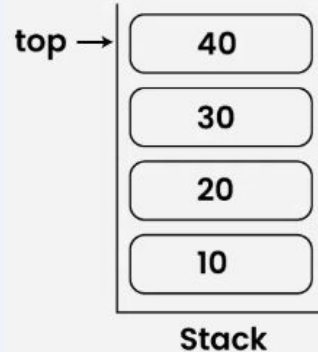
Stack

Commonly Used operations in stack

- empty()
- size()
- push(x)
- **pop()**
- top()

Only the top element of the stack can be deleted by using pop().

st1.**pop()**;



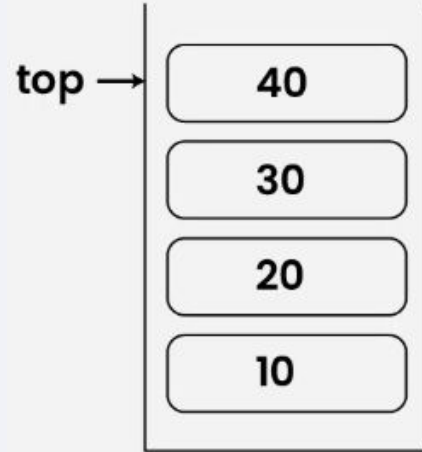
Stack

Commonly Used operations in stack

- `empty()`
- `size()`
- `push(x)`
- `pop()`
- `top()`

Only the top element of the stack can be accessed using `top()`

`st1.top();`



Top Element = 40

Stack


```

#include <iostream>
#include <stack> // Required for using the stack container
using namespace std;

int main() {
    // 1. Initialize an empty stack of integers
    stack<int> s;

    // Push elements: 10, 20, 30
    s.push(10);
    s.push(20);
    s.push(30);

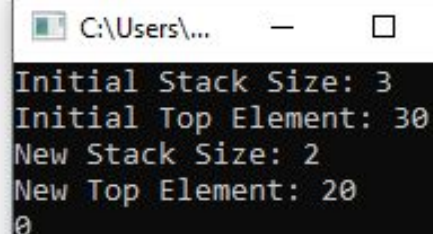
    // Current State Check
    cout << "Initial Stack Size: " << s.size() << endl; // Output: 3
    cout << "Initial Top Element: " << s.top() << endl; // Output: 30

    // pop(): Removes the top element (30) from the stack.
    s.pop();

    // State Check After Pop
    cout << "New Stack Size: " << s.size() << endl; // Output: 2
    cout << "New Top Element: " << s.top() << endl; // Output: 20

    // empty(): Checks if the stack has any elements.
    cout << s.empty() << endl; // Output: 0 (which means false)
    return 0;
}

```



```

C:\Users\...
Initial Stack Size: 3
Initial Top Element: 30
New Stack Size: 2
New Top Element: 20
0

```

Queue

A queue is a container adapter that stores elements in FIFO (First In, First Out) order.

Commonly Used Operations in Queue

- `empty()`
- `size()`
- `push(x)`
- `pop()`
- `front()`
- `back()`

Queue

Commonly Used operations in Queue

- **empty()**
- size()
- push(x)
- pop()
- front()
- back()

This checks whether the queue is empty.
It returns true if the queue has no elements;
otherwise, it returns false.

```
q1.empty();
```

Queue

Commonly Used operations in queue

- empty()
- size()
- push(x)
- pop()
- front()
- back()

Returns the number of elements currently in the queue.

```
q1.size();
```

Queue

Commonly Used operations in queue

- empty()
- size()
- **push(x)**
- pop()
- front()
- back()

In queue, new elements can only be inserted at back of the queue by using push(x) function (also called enqueue).

q1.**push**(8);



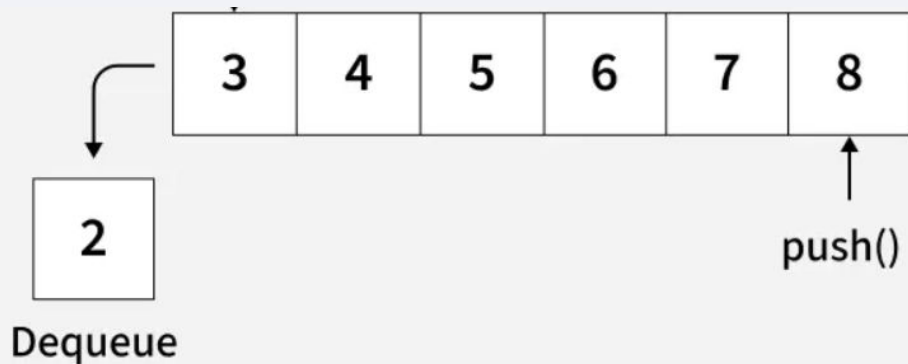
Queue

Commonly Used operations in queue

- empty()
- size()
- push(x)
- **pop()**
- front()
- back()

Elements can only be deleted from the front of the queue using the pop() function (also called dequeue).

q1.**pop()**;



Queue

Commonly Used operations in queue

- empty()
- size()
- push(x)
- pop()
- front()
- back()

Only the front and back elements of the queue can be accessed by using front() and back() functions respectively.

```
q1.front();  
q1.back();
```

```
#include <iostream>
#include <queue> // Required for using the queue container
using namespace std;

int main() {
    // 1. Initialize an empty queue of integers
    queue<int> q1;

    // push(x): Adds elements to the back of the queue
    q1.push(10); // 10 is the FIRST element in
    q1.push(20);
    q1.push(30); // 30 is the LAST element in

    // size(): Returns the current number of elements
    cout << "Queue size after PUSH: " << q1.size() << endl; // Output: 3

    // front(): Accesses the element at the front (10)
    cout << "Front element: " << q1.front() << endl; // Output: 10

    // back(): Accesses the element at the back (30)
    cout << "Back element: " << q1.back() << endl; // Output: 30

    // pop() removes the front element
    q1.pop(); // The queue after pop: 20, 30

    // Final empty() check
    cout << "Empty? " << (q1.empty() ? "Yes" : "No") << endl; // Output: No
    return 0;
}
```


Map

The **Map** is an associative container in the C++ Standard Template Library (STL). It stores elements in sorted order based on a unique **key**.

Commonly Used Operations in Map

- `size()`
- `clear()`
- `empty()`
- `begin()`
- `end()`
- `erase()`
- `insert()`
- `find()`
- `count()`

Map

Commonly Used member function in Map

- **size()**
- clear()
- empty()
- begin()
- end()
- erase()
- insert()
- find()
- count()

Returns the number of key-value pairs in the map.

```
m1. size();
```

```
map<int, string> m1 = {{1, "A"}, {2, "B"}};  
int count = m1.size(); // count will be 2
```

Map

Commonly Used member function in Map

- size()
- clear()
- empty()
- begin()
- end()
- erase()
- insert()
- find()
- count()

Removes all elements from the map.

```
m1. clear();
```

```
map<int, string> m1 = {{1, "A"}, {2, "B"}};  
m1.clear(); // m is now {}
```

Map

Commonly Used member function in Map

- size()
- clear()
- **empty()**
- begin()
- end()
- erase()
- insert()
- find()
- count()

Checks whether the map is **empty**.

```
m1. empty();
```

```
map<int, string> m1 = {{1, "A"}};  
bool check = m1.empty(); // check will be false  
m1.clear();  
bool check2 = m1.empty(); // check2 will be true
```

Map

Commonly Used member function in Map

- size()
 - clear()
 - empty()
 - **begin()**
 - **end()**
 - erase()
 - insert()
 - find()
 - count()
- begin() returns an iterator to the first element (smallest key).
 - end() returns an iterator to the sentinel position (immediately after the last actual item).
- ```
m1. begin();
m1. end();
```

# Map

## Commonly Used member function in Map

- size()
- clear()
- empty()
- begin()
- end()
- **erase(x)**
- insert()
- find()
- count()

Removes the element specified by a key or an iterator.

m1. **erase**(key);

```
map<int, string> m1 = {{1, "A"}, {2, "B"}};
// Removes the element with key 1
m1.erase(1); // m1 is now {{2, "B"}}
```

# Map

## Commonly Used member function in Map

- `size()`
  - `clear()`
  - `empty()`
  - `begin()`
  - `end()`
  - `erase()`
  - **`insert()`**
  - `find()`
  - `count()`
- Inserts a new key-value pair into the map. If the key already exists, the insertion is ignored (the map enforces unique keys).

`m1. insert(key, value);`

```
map<int, string> m1;
m1.insert({1, "Apple"}); // Inserts {1, "Apple"}
m1.insert({1, "Banana"}); // This insertion is ignored
// m1 is still {{1, "Apple"}}
```

# Map

## Commonly Used member function in Map

- size()
- clear()
- empty()
- begin()
- end()
- erase()
- insert()
- **find()**
- count()

Searches for the element with the specified key. If key is not present in the map, find() returns a special value called end(), meaning not found.

m1. **find**(key);

```
map<int, string> m1 = {{1, "A"}};
auto it = m1.find(1); // it points to {1, "A"}
auto it2 = m1.find(5); // it2 points to m1.end()
```



# Map

## Commonly Used member function in Map

- size()
- clear()
- empty()
- begin()
- end()
- erase()
- insert()
- find()
- **count()**

Count the occurrence of the given key in the map container.

```
m1. count(key);
```

```
m1.insert({2, 30}, {1, 40}, {3, 60});
```

```
int cnt1 = m1.count(1); //output : 1
```

```
int cnt2 = m1.count(9); //output : 0
```

# Map

## Printing all elements in the map using iterator

```
map<string, int> myMap;
myMap["Bangladesh"]=300;
myMap["Italy"]=50;
myMap["Canada"]=100;
myMap["America"]=200;

map<string, int>::iterator it;
for(it=myMap.begin(); it!=myMap.end(); it++){
 cout<<"Key : " << it->first;
 cout<<" and Element : "<<it->second<<endl;
}
```

# Set

Sets are containers that store unique elements following a specific order.

Commonly Used Operations in set

- `size()`
- `clear()`
- `empty()`
- `erase()`
- `insert()`
- `find()`
- `count()`
- `lower_bound()`
- `upper_bound()`

# Set

## Commonly Used member function in Set

- `size()`

- `clear()`

- `empty()`

- `erase()`

- `Insert()`

- `find()`

- `count()`

- `lower_bound()`

- `upper_bound()`

Returns the number of unique elements in the set.

`s1.size();`

```
set<int> s = {10, 20, 30};
int count = s.size(); // count will be 3
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- erase()
- Insert()
- find()
- count()
- lower\_bound()
- upper\_bound()

Removes all elements from the set, making the set empty.

s1. **clear**();

```
set<int> s = {10, 20};
s1.clear(); // s is now {}
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- **empty()**
- erase()
- Insert()
- find()
- count()
- lower\_bound()
- upper\_bound()

Checks whether the set is empty.

s1. **empty**();

```
set<int> s1 = {10};
bool check = s1.empty(); // check will be false
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- **erase(x)**
- Insert()
- find()
- count()
- lower\_bound()
- upper\_bound()

Removes the element specified by its value or an iterator.

s1. **erase**(x);

```
set<int> s1 = {10, 20, 30};
// Removes the element with value 20
s1.erase(20); // s is now {10, 30}
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- erase()
- **insert(x)**
- find()
- count()
- lower\_bound()
- upper\_bound()

Adds a new element to the set. If an element with the same value already exists, the set ignores the insertion because all elements must be unique.

s1. **insert**(x);

```
set<int> s1;
s1.insert(10); // s1 is {10}
s1.insert(20); // s1 is {10, 20}
s1.insert(10); // Ignored
```



# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- erase()
- insert()
- **find(x)**
- count()
- lower\_bound()
- upper\_bound()

Searches for the element with the specified value. Returns an iterator to the element if found; otherwise, it returns the s1.end() iterator.

s1. **find**(x);

```
set<int> s1 = {10};
auto it = s1.find(10); // it points to the element 10
auto it2 = s1.find(5); // it2 points to s1.end()
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- erase()
- insert()
- find()
- **count()**
- lower\_bound()
- upper\_bound()

Count occurrence of an element with the given value. Since set elements are unique, it returns either 1 (found) or 0 (not found)

s1. **count**(x);

```
set<int> s = {10, 20};
int num1 = s1.count(20); // num1 is 1 (found)
int num2 = s1.count(5); // num2 is 0 (not found)
```

# Set

## Commonly Used member function in Set

- `size()`
- `clear()`
- `empty()`
- `erase()`
- `insert()`
- `find()`
- `count()`
- **`lower_bound()`**
- `upper_bound()`

Returns an iterator pointing to the first element in the set that is not less than the given value (greater than or equal to).

```
s1.lower_bound();
```

```
set<int> s1 = {10, 20, 30};
auto it = s1.lower_bound(15); // it points to 20
auto it2 = s1.lower_bound(20); // it2 points to 20
```

# Set

## Commonly Used member function in Set

- size()
- clear()
- empty()
- erase()
- insert()
- find()
- count()
- lower\_bound()
- upper\_bound()

Returns an iterator pointing to the first element in the set that is strictly greater than the given value.

```
s1.upper_bound();
```

```
set<int> s1 = {10, 20, 30};
auto it = s1.upper_bound(20); // it points to 30
auto it2 = s1.upper_bound(5); // it2 points to 10
```

# Set

## Printing all elements in the set using iterator

```
set<char> mySet;
mySet.insert('d');
mySet.insert('a');
mySet.insert('b');
mySet.insert('e');

set<char>:: iterator it1;

for(it1=mySet.begin(); it1!=mySet.end(); it1++){
 cout<< *it1 <<" ";
}
cout<<endl;
```